

Weekly Canvas discussion posts — Deliverable 3

Grimdark Forge · COSC3046 · Augustus Zibell-Barnes (S4246254) · solo team

Each post is written to be pasted straight into the Canvas course discussion board. Where a post asks for peer feedback it is addressed to the other teams in the cohort, since there is no second team member. Dates follow the Semester 2 study weeks between the AT1 submission (14 August 2026) and the AT2 submission (18 September 2026).

Week 4 — 21 August 2026 — Kicking off AT2: planning the conversion

Progress. AT1 is in, so this week was planning rather than coding. I re-read the AT2 brief against the static site and made a list of everything that has to change:

- 23 `.html` pages become `.php` pages, but the wireframes and CSS stay as they are.
- The copy-pasted header/nav/footer becomes three PHP includes.
- Login/logout (username only) goes into the shared header so it is on every page.
- All the demo content in the pages has to move into hard-coded associative arrays.
- Every form needs JavaScript *and* PHP validation, and every `required` / `pattern` / `maxlength` attribute has to come out.

Decision. I am building the shared layer first (`shared/bootstrap.php` , header, nav, footer, validation engine) before touching any module, because all four modules need the same helpers and I do not want to write them four times. Being a solo team makes this easier — there is nobody blocked waiting on the shell — but it also means there is no one to catch my mistakes, so I am writing a scripted checker that renders every page and checks headings, labels, `alt` text and links automatically.

Question for other teams. How are you keeping data alive between requests without a database? PHP forgets everything at the end of a request, so "in-memory associative arrays" only work if you can park them somewhere. I am looking at APCu on the CoreTeaching servers but have not confirmed it is enabled — if anyone has checked `phpinfo()` on titan, I would appreciate knowing what you found.

Week 5 — 28 August 2026 — Shell, sessions and the in-memory store

Progress. The shared shell is done and every page now `require`s it. `gf_base()` works out the `../` prefix from `__DIR__` , so the same header file serves root pages and module pages. Login and logout work: `account.php` takes a username, stores it in `$_SESSION` , and the header switches between a *Log in* button and the avatar + *Log out* control.

The storage problem, solved. Following up on last week's question: APCu is not guaranteed on titan, so `shared/store.php` tries APCu first and otherwise falls back to a single serialised scratch file in the system temp directory under `flock(LOCK_EX)` . Either way the site code just calls `collection('posts')` and gets a reference to an

array; it never knows which backend it is on. The store is loaded once per request, and written back at shutdown only if the serialised hash changed, so read-only page views cost nothing. A *Reset demo data* button on the account page re-seeds it from `shared/data.php`.

I am aware the file fallback is technically a file — I have documented it as a known limitation and it is never queried like a database, it only holds the array structure.

Something I got wrong. My first version stored the logged-in user's cart in the session. The brief wants the cart saved *per user*, so it has moved into the store keyed by username, and a guest cart is merged into the account's cart on login (`gf_merge_guest_cart()`).

Feedback request. Would anyone be willing to log into <https://titan.csit.rmit.edu.au/~s4246254/welcome.php> as `kaya` and try the login/logout flow with only a keyboard? I want to know whether the focus order through the header feels right before I build the locale menu in the same spot.

Week 6 — 4 September 2026 — Validation in both layers without HTML attributes

Progress. Forum and blog CRUD are working end to end (create thread, reply, edit own post, soft-delete, and the equivalent for blog posts and comments). Reviews and shop are next.

Technical explanation: validation. The rule I set myself was "declare each rule once". Each PHP page has a `$rules` array, e.g.

```
'post-title' => ['label' => 'Post title', 'required' => true, 'min' => 3, 'max' => 90],
```

`shared/validation.php` runs those rules on the POST and returns `[field => message]`. The same rules are printed onto the controls as `data-required`, `data-minlength="3"` and `data-maxlength="90"`, and `assets/js/validate.js` reads them on submit. Both layers put the message in the same `#<name>-error` element and the same `#form-errors` summary, set `aria-invalid="true"` and `aria-describedby` on the failed control, and move focus to the summary — so the page looks identical whether the browser or the server caught the error. There are no HTML validation attributes anywhere; a grep for `required=|pattern=|maxlength=` across the templates is part of my pre-commit check.

What I would do differently. I mirrored the rules by hand in the templates at first and missed two. The fix was a small helper that emits the `data-*` attributes from the same `$rules` array, so they cannot drift.

Feedback request. If any team has a form with a validation dependency (e.g. "alt text is required only if an image is attached"), how did you express it client-side? I ended up with a `data-required-if-file` attribute and would like to compare approaches.

Week 7 — 11 September 2026 — Ownership, CSRF and the shop's money maths

Progress. All four modules now have working create/read/update/delete. The shop has cart lines with variant, quantity and note; a discount code (`FORGE10`); delivery options; checkout with card validation; and an order-confirmation page. Stock is re-checked and decremented at order time, not at add-to-cart.

Technical explanation: security handling. Three layers, all in `shared/bootstrap.php` so every module gets them for free:

1. *Input.* Every request value is read with `filter_input()` — `FILTER_VALIDATE_INT` for ids, `FILTER_DEFAULT` + `FILTER_REQUIRE_SCALAR` for strings — never `$_POST` directly.
2. *CSRF.* `csrf_field()` prints a hidden token (from `random_bytes(16)` , held in the session) into every POST form, and every handler calls `csrf_ok()` — a `hash_equals()` compare — before it touches the store.
3. *Output.* Everything echoed into HTML goes through `e()` (`htmlspecialchars` with `ENT_QUOTES`), including translated strings via `te()` .

Ownership is enforced server-side: `owns($author)` gates the Edit/Delete buttons, but the edit and delete pages check it again and answer HTTP 403 with a page explaining who wrote the content. Typing `forum/edit-post.php?id=1` while logged in as the wrong user gets the 403.

Card numbers are Luhn-checked, and only the last four digits are kept on the order record.

Money. Every price is an integer number of cents. `cart_totals()` computes subtotal, discount, delivery and GST (total ÷ 11) in PHP; `shop.js` only re-renders the same figures live as the quantity changes. Checked by hand: $208.50 - 20.85 = 187.65$, free delivery over 120.00.

Feedback request. Two markers are meant to be able to interact through the site. Could another team open a thread on my forum while I am logged in elsewhere and confirm it appears for me without a reset? I have tested it across two browsers on one machine but not across two machines.

Week 8 — 18 September 2026 — Locales, final checks and submission

Progress. Submitted. The last piece was the locale toggle agreed in the AT1 charter: a header button that expands a list of `en-AU` and `de-DE` , posts to `shared/locale.php` , stores the choice in the session and sets `<html lang>` on every page. All 747 UI strings pass through `t()` and are mapped in `shared/lang/de-DE.php` ; dates, prices and counts go through `fmt_date()` , `money()` and `number()` (PHP `intl` with a manual fallback) and are re-formatted in the browser with `Intl.DateTimeFormat` / `Intl.NumberFormat` . In German a price reads `42,00 AU$` and a date `13. August 2026 um 21:40` .

Verification before submitting.

- 23 pages × 2 locales rendered through PHP and checked for `<html lang>` , one `h1` , no heading jumps, labels on every control, `alt` on every image, no duplicate ids, no inline styles, no HTML validation attributes, every internal link resolving, no PHP warnings.

- CRUD for each module exercised over HTTP including invalid submissions and foreign-owner edit/delete requests (403).
- `php -l` on every file; no PHP 8-only syntax.

Known issues I have declared rather than hidden: the scratch-file store fallback; uploaded images held as data URIs (1 MB cap); replies re-attaching to the nearest visible ancestor after a soft delete; no moderator view of deleted forum posts yet; and the W3C validator / Lighthouse not yet run on the hosted copy.

Reflection on working solo. The shared-layer decision paid off — building the shell, validation and formatting helpers first meant the fourth module took about a third of the time of the first. The cost was the lack of a second pair of eyes; the scripted checks and the feedback from the threads above were my substitute for peer review. For AT3 I intend to keep the same approach: change the storage backend behind `collection()` and leave the modules alone.

Thanks to everyone who tried the site and replied over the past weeks.